

→ Previous video

- * Introduction to recursion on lists
- * Intro to Isabelle + proof / reasoning approaches revision

→ Current video

- * Induction on lists aka. structural induction

→ Intuition

- * Recall induction on numbers

- to prove something holds for all numbers, we prove

- I.H. 1. it holds for 0 → Base case
2. If it holds for a number "n", then we step case
 can show it holds for "n+1".

- Why does proving these two goals mean the property holds for any "n"?
 - for any "n", we show that $P_0, P_1, P_2, \dots, P_n$

Base
step
 - This works because any nat is made up of a finite applications of "suc"

* Analogy for finite lists

- we have that any list can be constructed with a finite number of "cons", $x_1 \mathbin{\&\&} x_2 \mathbin{\&\&} \dots \mathbin{\&\&} x_n \mathbin{\&\&} []$
- To prove a property holds for any list xs ,

goal₁: property holds for $[]$

goal₂: If the property holds for xs , then we can show it holds for $x \mathbin{\&\&} xs$, for any x .

→ formal induction principle

for nats: $\underbrace{P_0}_{\text{Base}} \quad \underbrace{P_n \Rightarrow P_{n+1}}_{\text{I.H.}} \quad \text{step}$

for lists: $\underbrace{P [] \quad P xs \Rightarrow \forall y. (y \mathbin{\&\&} xs)}_{P \text{ ocs}}$

→ Example

consider append $\left[\begin{array}{l} \text{append } [] \ ys = ys \\ \text{append } (x \mathbin{\&\&} xs) \ ys = x \mathbin{\&\&} (\text{append } xs \ ys) \end{array} \right.$

- let $xs @ ys$ denote $\text{append } xs \ ys$
- prove that $(xs @ ys) @ zs = xs @ (ys @ zs)$
 - I.E. append is assoc.

proof by induction on xs

Base case: $xs = []$

goal $([] @ ys) @ zs = [] @ (ys @ zs)$

- $([] @ ys) @ zs = ys @ zs$ by subst append_simps
- $[] @ (ys @ zs)$ by subst cr

Step case

assumptions

I.H. $(xs @ ys) @ zs = xs @ (ys @ zs)$

goal: $((x \mathbin{\&\&} xs) @ ys) @ zs = (x \mathbin{\&\&} xs) @ (ys @ zs)$

1. $((x \mathbin{\&\&} xs) @ ys) @ zs = (x \mathbin{\&\&} (xs @ ys)) @ zs$ by append_simps

$= x \mathbin{\&\&} ((xs @ ys) @ zs)$ by append_simps

$= x \mathbin{\&\&} (xs @ (ys @ zs))$ by I.H.

$= (x \mathbin{\&\&} xs) @ (ys @ zs)$ by append_simps